

COS 201: Computer Programming I

(3 Units C1: LH 30; PH 45)

LECTURER:

Engr. Dr. INWELEGBU J.O.

27/10/2025

Learning Outcomes

- ❖ At the end of this course, students should be able to:
- ❖ 1. identify different programming paradigms and their approaches to programming;
- ❖ 2. write programmes using basic data types and strings;
- ❖ 3. design and implement programming problems using selection;
- ❖ 4. design and implement programming problems using loops;
- ❖ 5. use and implement classes as data abstractions in an object-oriented approach;
- ❖ 6. implement simple exception handling in programmes;
- ❖ 7. develop programmes with input/output from text files; and
- ❖ 8. design and implement programming problems involving arrays.

Course Contents

- ❖ Introduction to computer programming
 - ❖ Functional programming
 - ❖ Declarative programming
 - ❖ Logic programming
 - ❖ Scripting languages
 - ❖ Introduction to object-orientation as a technique for modelling computation.
 - ❖ Introduction of a typical object-oriented language, such as Java.
 - ❖ Basic data types, variables, expressions, assignment statements and operators.
 - ❖ Basic object-oriented concepts: abstraction; objects; classes; methods; parameter passing; encapsulation.
 - ❖ Introduction to Strings and string processing.
 - ❖ Simple I/O; control structures; Arrays; Simple recursive algorithms; inheritance; polymorphism.
- 

Computer Programming?

- What is programming?
 - The process of writing, testing, and maintaining the source code of computer programs.
 - Telling the computer what to do
 - Why learn to program?
 - Develops logic and problem-solving skills
 - Improves attention to detail
 - It's fun!
- 

INTRODUCTION TO COMPUTER PROGRAMMING

Engr. Dr. INWELEGBU J.O

Introduction to Programming

Program: The term program refers to a set of instructions that instructs a computer on what to do. Programs are Solutions to Problems and they are written using programming languages.

- A program can instruct a computer to:
 - Read/ accept Input data
 - Calculate or compare
 - Store data
 - Write or display Output and communication messages.

The term **software** refers to a computer program or set of programs and its associated documentation such as user guide, technical manual

Programming Languages

- **Programming:** This refers to the process of writing computer programs using a programming language.
- The person who write such instructions is a programmer.

Programming Languages

- Each natural language has a systematic method of using symbols of a language. This is dictated by rules of grammar – semantic (structure) and syntax (words and symbols)
- Similarly, computer programming languages are governed by the structure and syntax.
- In natural languages one can break the syntax rule such but we can use inference (guesswork) to get what the person means.
- However, computer being a machine are receptive to only the exact syntax rules of the language being used.

Types of Programming Languages

- A **programming Language** - is a set of rules that provides a way of writing instructions that instructs a computer on what operations to perform.
- Programming languages can be classified into two broad categories namely:
 1. Low level languages further classified into two generations namely:

First Generation- Machine languages

Second Generation -Assembly languages

Types of Programming Languages

2. **High-level languages** further classified into three generations namely:

Third Generation- Procedural languages

Fourth Generation - Problem-oriented languages

Fifth Generation - Natural languages

Low Level Languages

MACHINE LANGUAGES (1GL)

- This is the oldest form of computer programming. Plug boards were used to represent data in binary (machine) form in the computer circuitry. Data represented in 1s and 0s are said to be written in **machine language**. For example, the code below represent a segment of a machine code:
 1011010100
 1010111101
 1000100100
- Machine language also varies according to make of computer – another characteristic that make them hard to work with.

ASSEMBLY LANGUAGES (2GL)

Assembly language use mnemonics such as ADD that are automatically converted to the appropriate sequence of 1s and 0s by a translator called **assembler**. Compared to machine languages, assembly languages are much easier for humans to understand and to use. The machine language code we gave above could be expressed in assembly languages as:

```
ADD    210(8,13),02B(4,7)
PACK   218(8,13),02F(4,7)
MP     212(6,13),21D(3,13)
```

Advantages of low level programming languages

1. They are executed very fast since no translator or minimum translation is involved.
2. They require very little memory space.
3. They have a high level of security since a program written in machine language is difficult to change or alter.

Disadvantages of low-level programming languages

1. Machine languages are time-consuming because the binary code instructions are complex.
2. They are difficult to understand.
3. They are difficult to debug and hence difficult to maintain.
4. They are not portable i.e can only be used by specific computers.

High level languages

PROCEDURAL LANGUAGES (3GL)

- Procedural languages, also known as third generation languages uses human like language. They are intended to solve general problems. Examples of procedural languages are C, Pascal, FORTRAN, COBOL etc.
- Like assembly languages, procedural languages must be translated into machine language so that the computer processes it. Depending on the language this translation is performed by either a compiler or an interpreter.

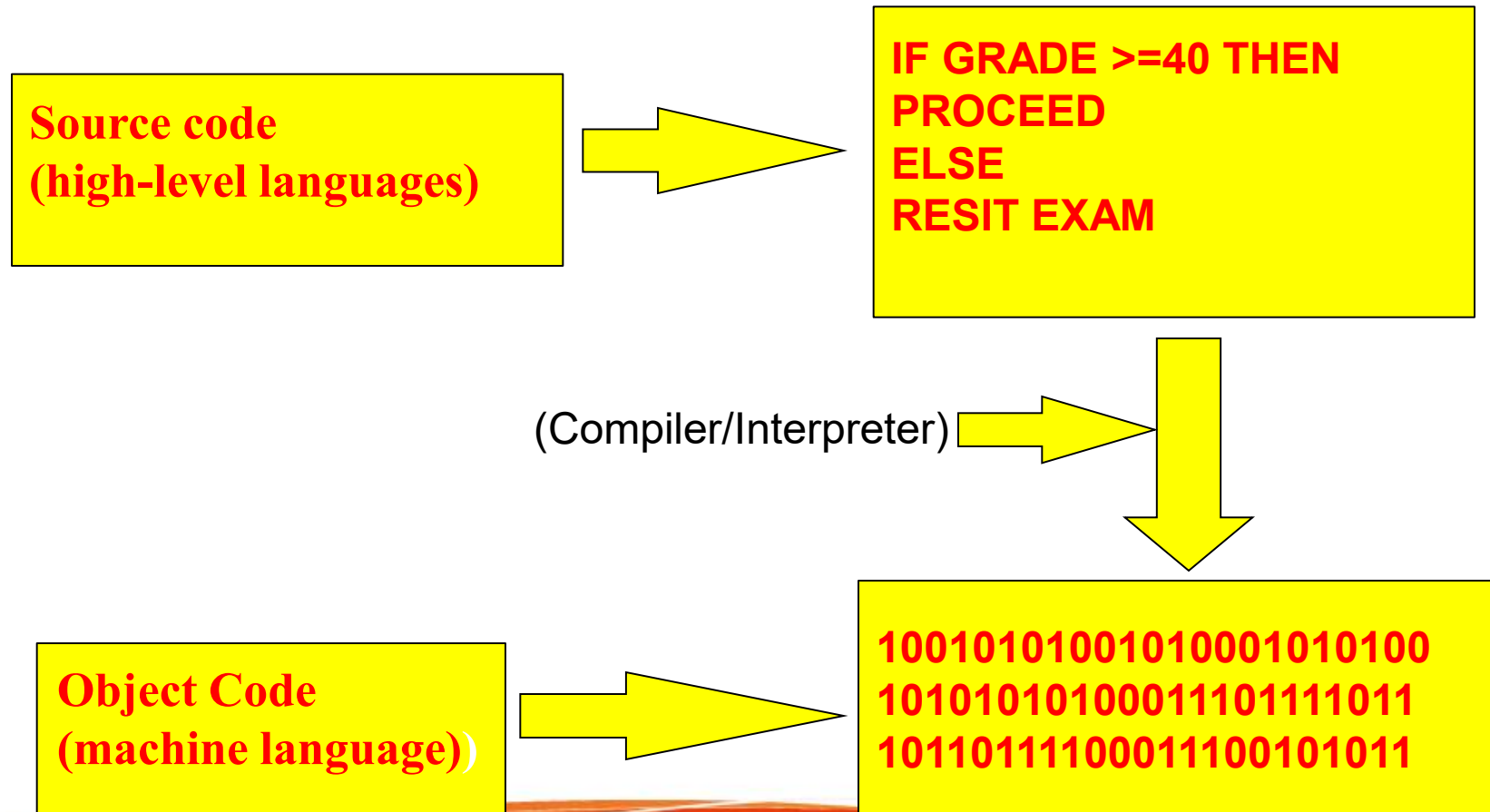
High level languages-Translators

- **A compiler** converts the programmer's procedural language program, called the **source code**, into a machine language called the **object code**. This object code can then be saved and run later. Examples of procedural languages using compilers are the standard version of PASCAL, COBOL, and FORTRAN.
- **An interpreter** converts the Procedural languages one statement at a time into machine code just before it is to be executed. No object code is saved. An example of procedural language using an interpreter is the standard version of BASIC.

Sample C++ Program

```
#include<iostream>
#include<math.h>
#define pi 3.149271
using namespace std;
int main()
{
    float area,r;
    cout.setf(ios::fixed);
    cout.setf(ios::showpoint);
    cout.precision(10);
    cout<<"Please enter the radius of the circle\n";
    cin>>r;
    area=pi*pow(r,2);
    cout<<"The area of the circle is: "<<area<<"\n";
    return 0;
}
```

Translation process



PROBLEM-ORIENTED LANGUAGES (4GL)

- Unlike general purpose languages, problem-oriented languages are designed to solve specific problems. While 3rd GLs focus on procedures and how a program will accomplish a specific task, 4th GLs are non-procedural and focus on specifying what the program is to accomplish. 4th GLs are more English-like, easier to program, and widely used by non-programmers . Some of these 4thGLs, are used for very specific applications. 4th GLs includes query languages e.g. SQL and application generators eg Visual Basic

NATURAL PROGRAMMING LANGUAGES

(5 GL)

- As they have evolved through the generations, computer languages have become more human-like. Clearly, the fourth generation query languages using commands that include words like SELECT, FROM and WHERE, is much more human-like than the 0s and 1s of machine languages. However, 5th GLs are still long way from natural languages such as English and Spanish that people use.
- The standard definition of a 5th GL is a computer language that incorporates the concepts of artificial intelligence to allow direct human communication. Examples are PROLOG, LISP etc. Additionally, these languages would enable a computer to run and to apply new information as people do. Rather than coding by keying in specific commands we would communicate more directly to a computer using natural languages or human languages such as English or Spanish.

NATURAL PROGRAMMING LANGUAGES

(5 GL)

Consider the following natural language statement that might appear in a 5th GL program for recommending medical treatment:

Does Bill like chips?;
likes(bill, chips).

Recently, the definition 5th GL has been expanded to include Visual Programming Languages such as Visual PROLOG that provide a natural interface for program development. This interface provides intuitive icons, menus and drawing tools for creating program code.

Advantages of high level languages

1. English-like phrasing of high level languages makes them easy to understand.
2. High level languages are portable i.e They are transferable from one computer to another.
3. They are user-friendly and easy to use and learn.
4. They are more flexible, hence they enhance creativity of the programmer and increase productivity in the work place.
5. High level languages are far much more easy to debug.

Disadvantages of high level languages

1. Their nature encourages loading of instructions in a word or statement hence the complexity of these instructions cause slower program processing.
2. They have to be interpreted or compiled for translation to machine language before the computer can trace them.

Survey of Programming Languages

C Lecture 1 : Getting Started: in C

Copyright © Engr. Dr. Inwelegbu
J.O (2025)
All Rights Reserved.

Objective

- Intro to C
 - Tools we will use
 - Program file structure
 - Variables
 - Read from screen and print to screen
 - Decisions (If)
- 

C Orientation

- Created in 1972 to write operating systems (Unix in particular)
 - By Dennis Ritchie
 - Bell Labs
 - Evolved from B
 - Can be portable to other hardware (with careful design – use Plauger's The Standard C Library book)
 - Built for performance and memory management – operating systems, embedded systems, real-time systems, communication systems
- 

C Standardization

- 1989 ANSI and ISO -> Standard C
 - 1999 C99
 - 2011 C11
-
- Don't get thrown when you lookup information on websites and find conflicts based upon standards

Later Languages

- 1979 C++ by Bjarne Stroustrup also at Bell
 - Object orientation
 - 1991 Java by Sun
 - Partial compile to java bytecode: virtual machine code
 - Write once, run anywhere
 - Memory manager – garbage collection
 - Many JVMs written in C / C++
- 

A First Program

```
#include <stdio.h>
int main(void)
{
    printf("This is my first C program.\n");
    return(0);
}
```

makes input and output available to us

header

statements

open and close braces mark the beginning and end

A First Program – What Does It Do?

```
printf("This is my first C program.\n");  
return(0);
```

Prints the message

`This is my first C program.`

Ends the program

Ends the line



Java Reminder

Program	C	Java
hello, world	#include<stdio.h>	public class HelloWorld {
	int main(void) {	public static void main(String[] args) {
	printf("Hello\n");	System.out.println("Hello");
	return 0;	}
	}	}

C Program Phases

- Editor - code by programmer
- Compiling using gcc:
 - Preprocess – expand the programmer's code
 - Compiler – create machine code for each file
 - Linker – links with libraries and all compiled objects to make executable
- Running the executable:
 - Loader – puts the program in memory to run it

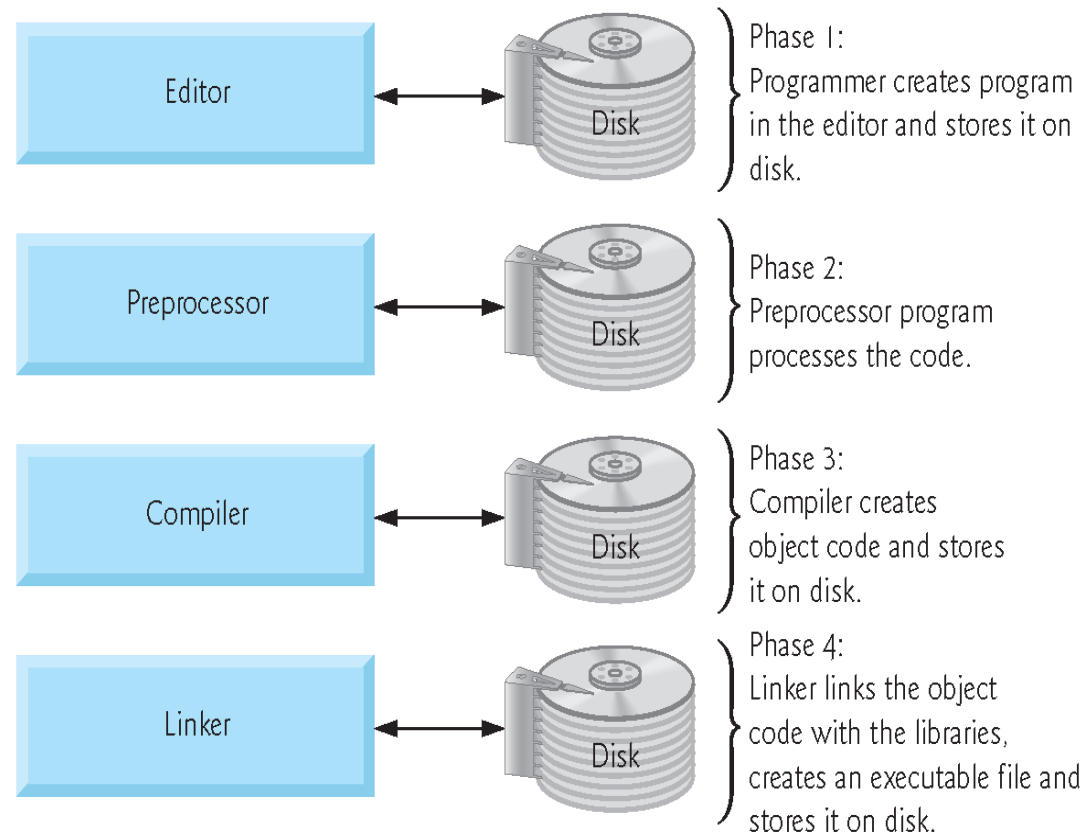


Fig. 1.7 | Typical C development environment. (Part I of 3.)

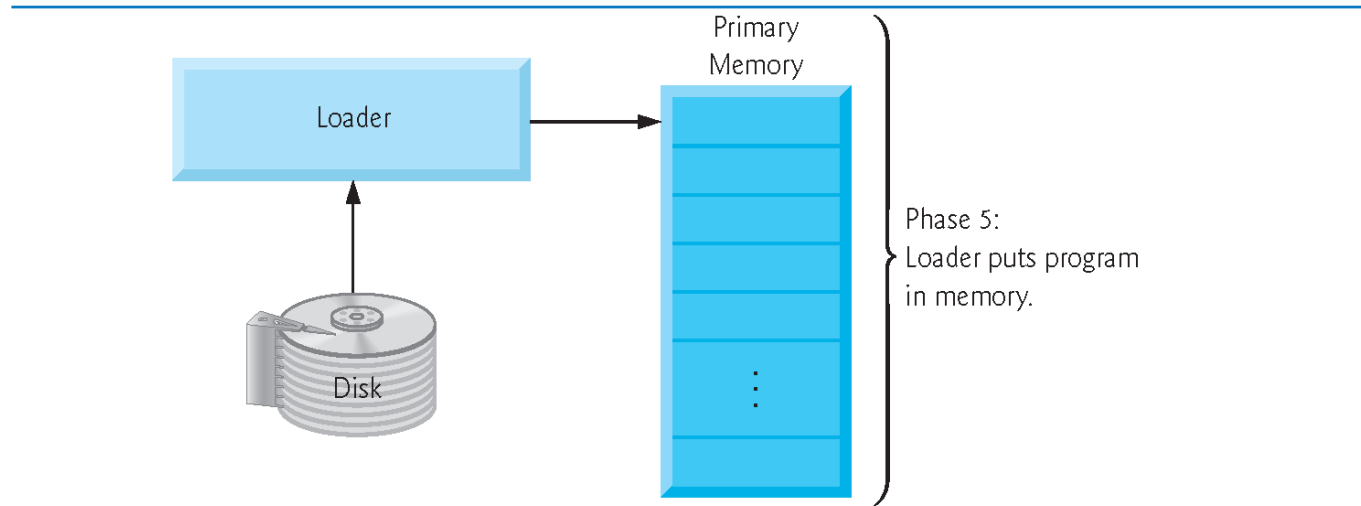


Fig. 1.7 | Typical C development environment. (Part 2 of 3.)

Run First Program

- Write in notepad++
 - Transfer with Filezilla
 - Connect to panther as terminal (putty) using SSH (Secure Shell)
 - More filename to see the file
 - gcc filename -o filename without c -g (ex: gcc hello.c -o hello -g)
 - ./hello
- 

Using variables

```
#include    <stdio.h>
int    main(void)
{
    int        sum, value1, value2, value3;
    float    average;
    value1 = 2;
    value2 = 4;
    value3 = 6;
    sum = 2 + 4 + 6;
    average = sum / 3;
    printf("The average  of %d , %d, %d is %f\n",
value1, value2, value3, average);
    return(0);
}
```

*Print a float value from the
rest of the parameter list*

Variables and Identifiers

- Variables have names – we call these names **identifiers**.
 - An identifier must begin with a letter or an underscore –
 - C is case sensitive upper case (capital) or lower case letters are considered different characters. **Average**, **average** and **AVERAGE** are three different identifiers.
 - Numbers can also appear after the first character.
 - However, C only considers the first 31 (external identifiers) or first 63 (internal identifiers) significant.
 - Identifiers cannot be reserved words (special words like **int**, **main**, etc.)
- 

User Input

- Let's rewrite the average program so it can find the average any 3 numbers we try:
 - We now need to:
 1. Find our three values
 2. Add the values
 3. Divide the sum by 3
 4. Print the result
- 

Average3.c

```
#include    <stdio.h>
```

```
int         main(void)
{
```

```
    int     value1, value2, value3;
    float   sum, average;
```

```
    printf("What is the first value? ");
```

```
    scanf("%d", &value1);
```

Read

The address of variable value1

```
    printf("What is the second value? ");
```

```
    scanf("%d", &value2);
```

*Indicates that we are
reading an integer*

```
printf("What is the third value? ");  
scanf("%d", &value3);
```

↑ *scanf needs the &
before the identifier*

```
sum = value1 + value2 + value3;  
average = sum / 3;
```

```
    printf("The average  of %d , %d, %d is  
%f\n", value1, value2, value3, average);  
    return(0);  
}
```


Scanf Conversion Characters

- <https://wpollock.com/CPlus/PrintfRef.htm#scanfConv>
Doubles on our machine are read with a lf. (A double is a long float.)

Formatting %d and %f

- The specifiers %d and %f allow a programmer to specify how many spaces a number will occupy and how many decimal places will be used.
- %*n*d will use at least *n* spaces to display the integer value in decimal (base 10) format.
- %*w*.*n*f will use at least *w* spaces to display the value and will have exactly *n* decimal places.
- Example:
 - `printf("The average of %2d , %2d, %2d is %5.2f\n", value1, value2, value3, average);`

Changing the width

Number	Formatting	Print as:
182	%2d	182
182	%3d	182
182	%5d	`182
182	%7d	'''182
-182	%4d	-182
-182	%5d	`-182
-182	%7d	'''-182

Changing the width (continued)

Number	Formatting	Print as:
23	%1d	23
23	%2d	23
23	%6d	23
23	%8d	23
11023	%4d	11023
11023	%6d	.11023
-11023	%6d	-11023
-11023	%10d	-11023

Changing The Precision

Number	Formatting	Prints as:
2.718281828	%8.5f	` 2.71828
2.718281828	%8.3f	` ` ` 2.718
2.718281828	%8.2f	` ` ` ` 2.72
2.718281828	%8.0f	` ` ` ` ` ` ` ` 3
2.718281828	%13.11f	2.71828182800
2.718281828	%13.12f	2.718281828000

Average - add comments

```
#include          <stdio.h>

/*
 * This program calculates average pay
 */

int              main(void)
{
    int          value1, value2, value3;
    float sum, average;
    string
    // now get the first value
    ;
```

comments

Character Data

- All of our programs so far have used variables to store numbers, not words.
- We can store one or more characters by writing:
`char x, s[10];`
 - `x` can hold one and only one character
 - `s` can hold up to nine characters (reserving 1 for ending null)
- For now, we use character data for input and output only.

A program that uses a character variable

```
#include    <stdio.h>

/* A very polite program that greets you by name */
int    main(void)
{
    char        name[25];

    /* Ask the user his/her name */
    printf("What is your name ? ");
    scanf("%s", name);

    /* Greet the user */
    printf("Glad to meet you, %s\n.", name);
    return(0);
}
```



Features so far

- Include
- Variable types: int, float, char
- Read using scanf
 - requires & for address of variable being read
- Print using printf
- Format strings: %f (float), %d (int), %u (unsigned int), %c (char), %s (character array)
- Comments /* */ or //

if and ***if-else*** and ***if-else if - else***

```
If (boolean_expression 1)
{ /statements }
  else if ( boolean_expression 2)
{ /* statements */ }
  else if ( boolean_expression 3)
  { /* statements */ }
else
{ /* statements */ }
```

IsItNeg.c - illustrate if

```
#include      <stdio.h>

// Tell a user if a number is negative

int    main(void)
{ float number;

    /* Ask the user for a number */

    printf("Please enter a number ? ");

    scanf("%f", &number);

    // Print whether the number is negative or not

    if (number < 0){

        printf("%f is a negative number\n",  number); }

    else {

        printf("%f is NOT a negative number\n",  number); }

    return(0); }
```

Relational operators

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
==	equals	$x == y$
!=	is not equal to	$1 != 0$
>	greater than	$x+1 > y$
<	less than	$x-1 < 2*x$
>=	greater than or equal to	$x+1 >= 0$
<=	less than or equal to	$-x + 7 <= 10$

Integer Division

- Our compound interest program prints the values for every year where every ten or twenty years would be good enough.
- What we really want to print the results only if the year is ends in a 5. (The remainder from division by 10 is 5).

Integer Division Results

$8 / 3 = 2$	$8 \% 3 = 2$
$2 / 3 = 0$	$2 \% 3 = 2$
$49 / 3 = 16$	$49 \% 3 = 1$
$49 / 7 = 7$	$49 \% 7 = 0$
$-8 / 3 = -2$	$-8 \% 3 = -2$
$-2 / 3 = 0$	$-2 \% 3 = -2$
$-2 / -3 = 0$	$-2 \% -3 = -2$
$2 / -3 = 0$	$2 \% -3 = 2$
$-49 / 3 = -16$	$-49 \% 3 = -1$

Choosing Data Types

- Sizes implementation dependent in limits.h
 - int -2147483648 to 2147483647
 - short -32768 to 32767
 - long -9223372036854775808 to 9223372036854775807
 - Float 1.17×10^{-38} to $3.4 * 10^{38}$
- Keyword unsigned starts at 0 but goes higher

Declaring Constants

- There are two ways of defining constants in C: using `#define` and `const`.
- `#define` is a compiler preprocessor which replaces each occurrence of the constant's name with its value:
- The general form of the constant declaration is:
`#define` *ConstantName* *ConstantValue*

- Let's take a look at a few examples:

```
#define    withholding_rate    0.8
#define    prompt    'y'
#define    answer    "yes"
#define    maxpeople 15
#define    inchperft 12
#define    speed limit    55
```


Declaring Constants

- The general form of the constant declaration is:

```
const datatype ConstantName =  
                                ConstantValue,  
    AnotherConstantName =  
                                AnotherConstantValue;
```

- Let's take a look at a few examples of constants:

```
const float    withholding_rate = 0.8;  
const char    prompt = 'y',  
                answer[] = "yes";  
  
const int     maxpeople = 15,  
                inchperft = 12;  
                speed_limit = 55;
```

Java Comparison Thus Far

Feature	C	Java
type of language	function oriented / imperative	object oriented
file naming conventions	stack.c, stack.h	Stack.java - file name matches name of class
basic programming unit	function	class / Abstract Data Type
portability of source code	possible with discipline	yes
portability of compiled code	no, recompile for each architecture	yes, bytecode is "write once, run anywhere"
compilation	gcc hello.c creates machine language code	javac Hello.java creates Java virtual machine language bytecode
buffer overflow	segmentation fault, core dump, unpredictable program	checked run-time error exception
boolean type	use int: 0 for false, nonzero for true OR include <stdbool.h> and use bool	boolean is its own type - stores value true or false
character type	char is usually 8 bit ASCII	char is 16 bit UNICODE
strings	'\0'-terminated character array	built-in immutable String data type
accessing a library	#include <stdio.h>	import java.io.File;

More Java Comparison

Feature	C	Java
printing to standard output	<code>printf("sum = %d", x);</code>	<code>System.out.println("sum = " + x);</code>
formatted printing	<code>printf("avg = %3.2f", avg);</code>	<code>System.out.printf("avg = %3.2f", avg)</code>
reading from stdin	<code>scanf("%d", &x);</code>	<code>int x = StdIn.readInt();</code>
declaring constants	<code>const</code> and <code>#define</code>	<code>final</code>
for loops	<code>for (i = 0; i < N; i++)</code>	<code>for (int i = 0; i < N; i++)</code>
variable auto-initialization	not guaranteed	instance variables (and array elements) initialized to 0, null, or false, compile-time error to access uninitialized variables
casting	anything goes	checked exception at run-time or compile-time
demotions	automatic, but might lose precision	must explicitly cast, e.g., to convert from long to int
variable declaration	at beginning of a block	before you use it
variable naming conventions	<code>sum_of_squares</code>	<code>sumOfSquares</code>

Credit: <http://introcs.cs.princeton.edu/java/faq/c2java.html>

Summary

- Tools we will use
 - Notepad++
 - Filezilla
 - Panther (gcc)
 - Putty
- Program file structure
 - #include <> or “ “
 - Main function

Summary Cont.

- Variables
 - int, float, char
 - unsigned keyword
 - String defined as char array : char name[26]
 - For bool, include stdbool.h
 - Constant:
 - #define name value
 - const type name = ?
 - Get address of variable with &
 - Cast with (type) var

Summary Cont.

- Read from screen and print to screen
 - Scanf (control string, variable addresses)
 - Printf(string, variables to insert)
 - Format strings %2f, %d, %s, %u
 - #include <stdio.h>
- Decisions
 - If / else if / else